Graphic Mode

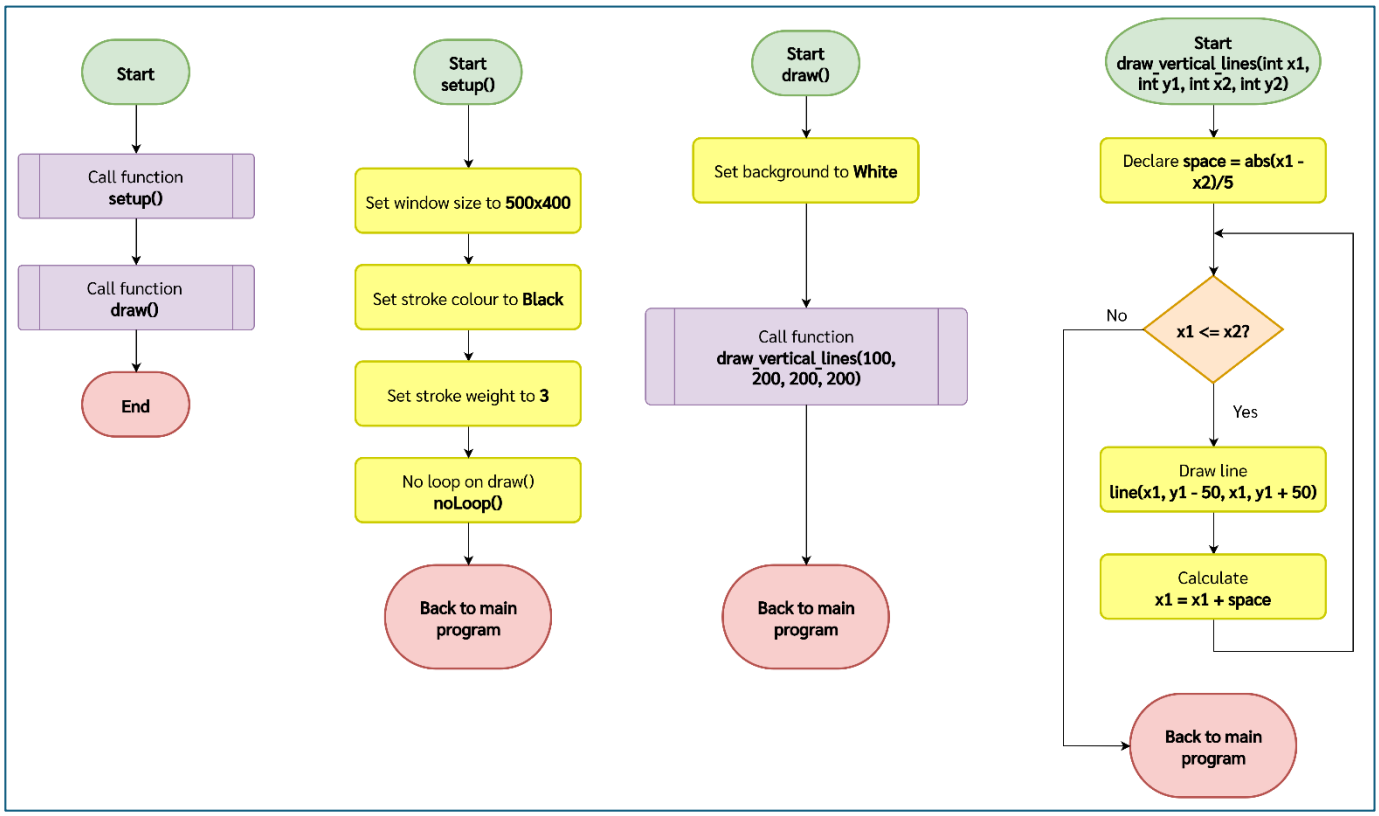
เครื่องมือที่ใช้: Processing V4.0a4

ข้อที่ 1 ทดลองวาดรูปและสร้างแอนิเมชันโดยใช้ตัวอย่างจาก lecture slides และสังเกตผลที่ได้

จากสไลด์หัวข้อ draw_vertical_lines การวาดกลุ่มเส้นตรงแนวตั้งเรียงขนานกันตามช่วงพิกัดที่กำหนด โดยใช้ฟังก์ชันย่อย draw_vertical_lines ที่รับค่าพิกัดจุดเริ่มต้นและจุดสิ้นสุดเข้ามาคำนวณระยะห่างระหว่างเส้นผ่านการหาผลต่างของพิกัดแกน X แล้วแบ่งออกเป็นห้าส่วนเท่ากัน ทำให้ได้ช่วงห่างคงที่สำหรับการจัดวางตำแหน่งเส้นตรง

ในกระบวนการวาดเส้น การประมวลผลถูกควบคุมด้วยคำสั่งวนซ้ำ while ซึ่งจะตรวจสอบเงื่อนไขตำแหน่งพิกัดแกน X ตั้งแต่จุดเริ่มต้นจนถึงจุดสิ้นสุด โดยในแต่ละรอบของการวนซ้ำ คำสั่ง line จะสร้างเส้นตรงแนวตั้ง ณ ตำแหน่งแกน X ปัจจุบัน โดยใช้อ้างอิงพิกัดแกน Y หลักในการขยายความยาวขึ้นด้านบนและลงด้านล่างทิศทางละ 50 พิกเซล เกิดเป็นเส้นตั้งที่มีความยาวรวม 100 พิกเซล จากนั้นโปรแกรมจะสะสมค่าพิกัดแกน X เพิ่มขึ้นตามระยะห่างที่คำนวณไว้ ทำให้เกิดการวาดเส้นตรงแนวตั้งจำนวน 6 เส้นเรียงตัวขนานกันด้วยระยะห่างที่สมดุลตลอดช่วงพิกัดที่กำหนด

(สามารถปรับตัวหารของ ตัวแปร space เพื่อเพิ่มจำนวนเส้นได้ และปรับ จุดเริ่มต้นและจุดสิ้นสุด เพื่อให้รูปกว้างขึ้นได้เช่นกัน)

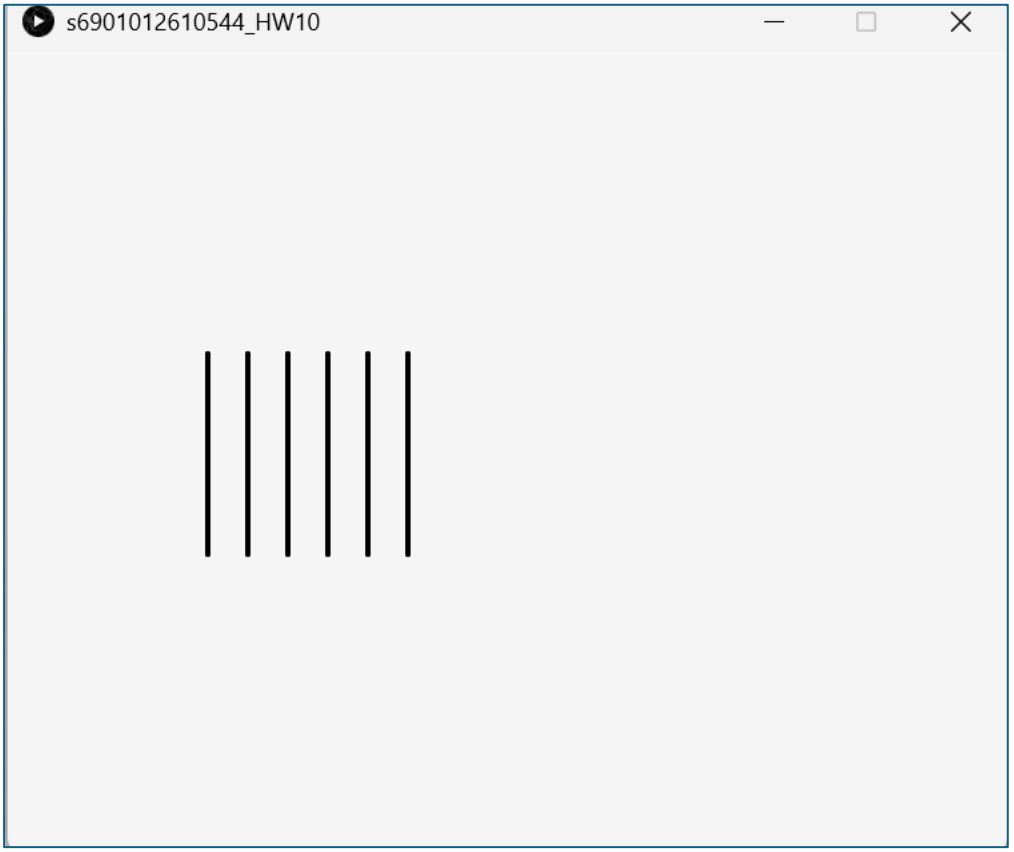


```
def setup():
    size(500, 400)
    stroke(0)
    strokeWeight(3)
    noLoop()

def draw():
    background(245)
    draw_vertical_lines(100, 200, 200, 200)

def draw_vertical_lines(x1, y1, x2, y2):
    space = abs(x1 - x2)/5

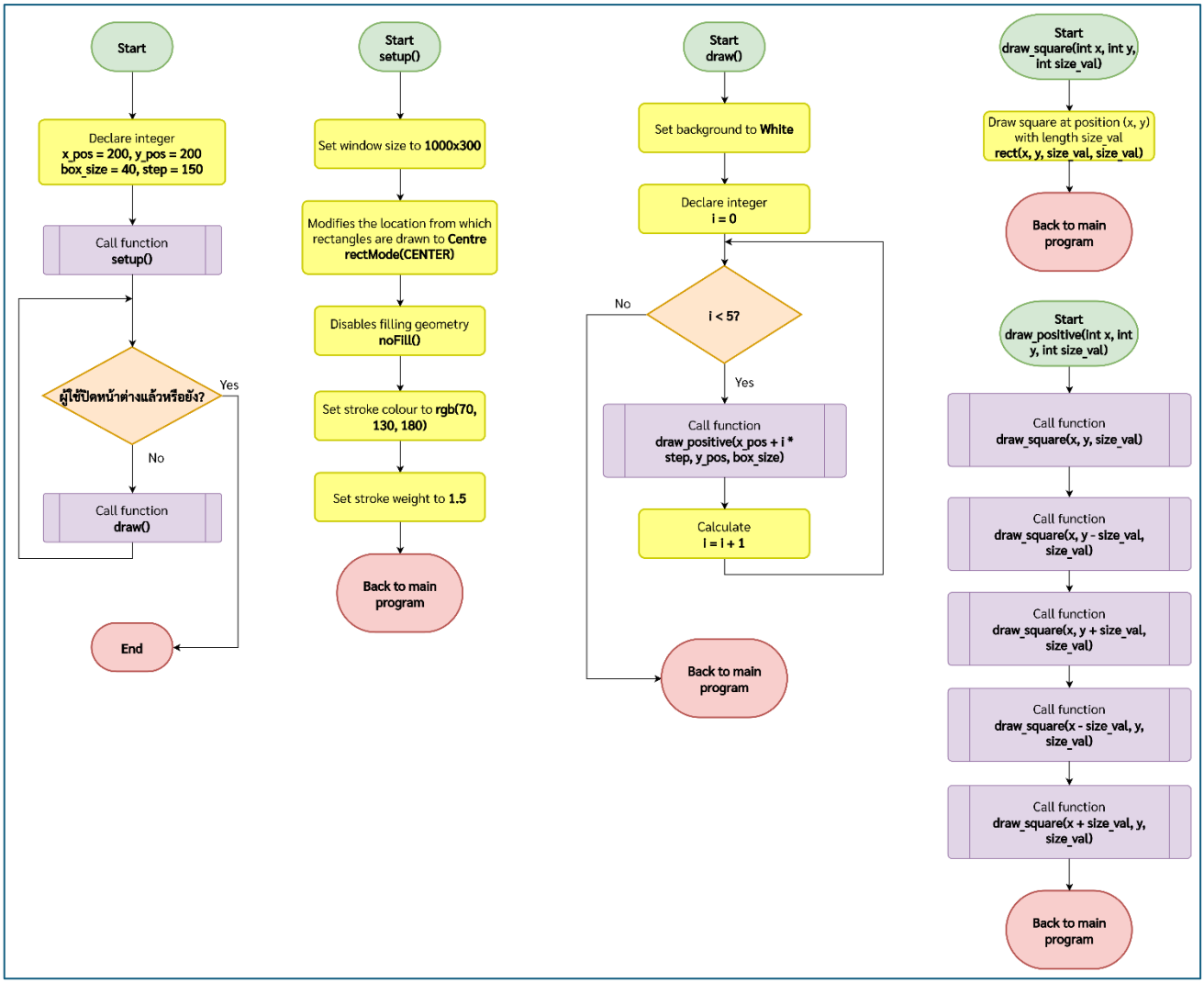
    while x1 <= x2:
        line(x1, y1 - 50, x1, y1 + 50)
        x1 = x1 + space
```



ข้อที่ 2 ใช้ draw_positive() จาก lab 3, เพื่อวาดรูปโดยใช้คำสั่งวนซ้ำ

การสร้างรูปทรงเครื่องหมายบวกครั้งนี้ โครงสร้างถูกแบ่งออกเป็นสองระดับ โดยมีฟังก์ชัน draw_square ทำหน้าในการสร้างรูปสี่เหลี่ยมจัตุรัสตามพิกัดและขนาดที่กำหนด จากนั้นฟังก์ชัน draw_positive จะทำหน้าที่จัดวางรูปสี่เหลี่ยมจัตุรัสจำนวนห้ารูปเข้าด้วยกันเพื่อประกอบเป็นเครื่องหมายบวก โดยอาศัยพิกัดจุดศูนย์กลาง (x, y) ที่ได้รับเข้ามา แล้วคำนวณตำแหน่งขยับออกไปในสี่ทิศทางด้วยระยะทางเท่ากับขนาดของสี่เหลี่ยม

สำหรับการจัดวางตำแหน่งรูปทรงซ้ำบนหน้าจอ ฟังก์ชัน draw และดำเนินการวนซ้ำผ่านคำสั่ง while จำนวนห้ารอบ เพื่อคำนวณพิกัดแกน X ใหม่ในแต่ละรอบด้วยการนำดัชนี i คูณกับระยะห่างที่กำหนดไว้ แล้วนำไปบวกเข้ากับพิกัดเริ่มต้น ส่งผลให้จุดศูนย์กลางของเครื่องหมายบวกในแต่ละรูปขยับไปทางขวาอย่างมีระเบียบ ในขณะที่ตำแหน่งแกน Y ยังคงถูกกำหนดไว้ที่ระดับเดิมตลอดการประมวลผล



```
#Global variables
x_pos = 200
y_pos = 200
box_size = 40
step = 150

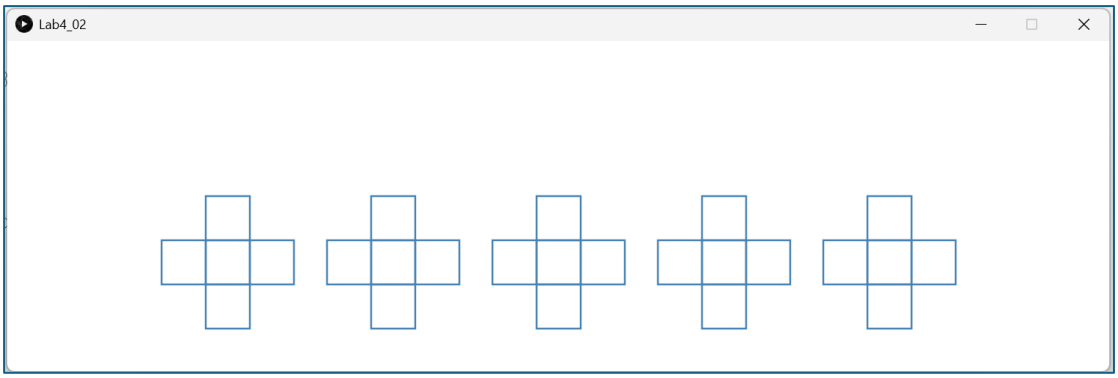
def setup():
    size(1000, 300)
    rectMode(CENTER)
    noFill()
    stroke(70, 130, 180)
    strokeWeight(1.5)

def draw():
    global x_pos, y_pos, box_size, step
    background(255)
    i = 0

    while i < 5:
        draw_positive(x_pos + i * step, y_pos, box_size)
        i = i + 1
```

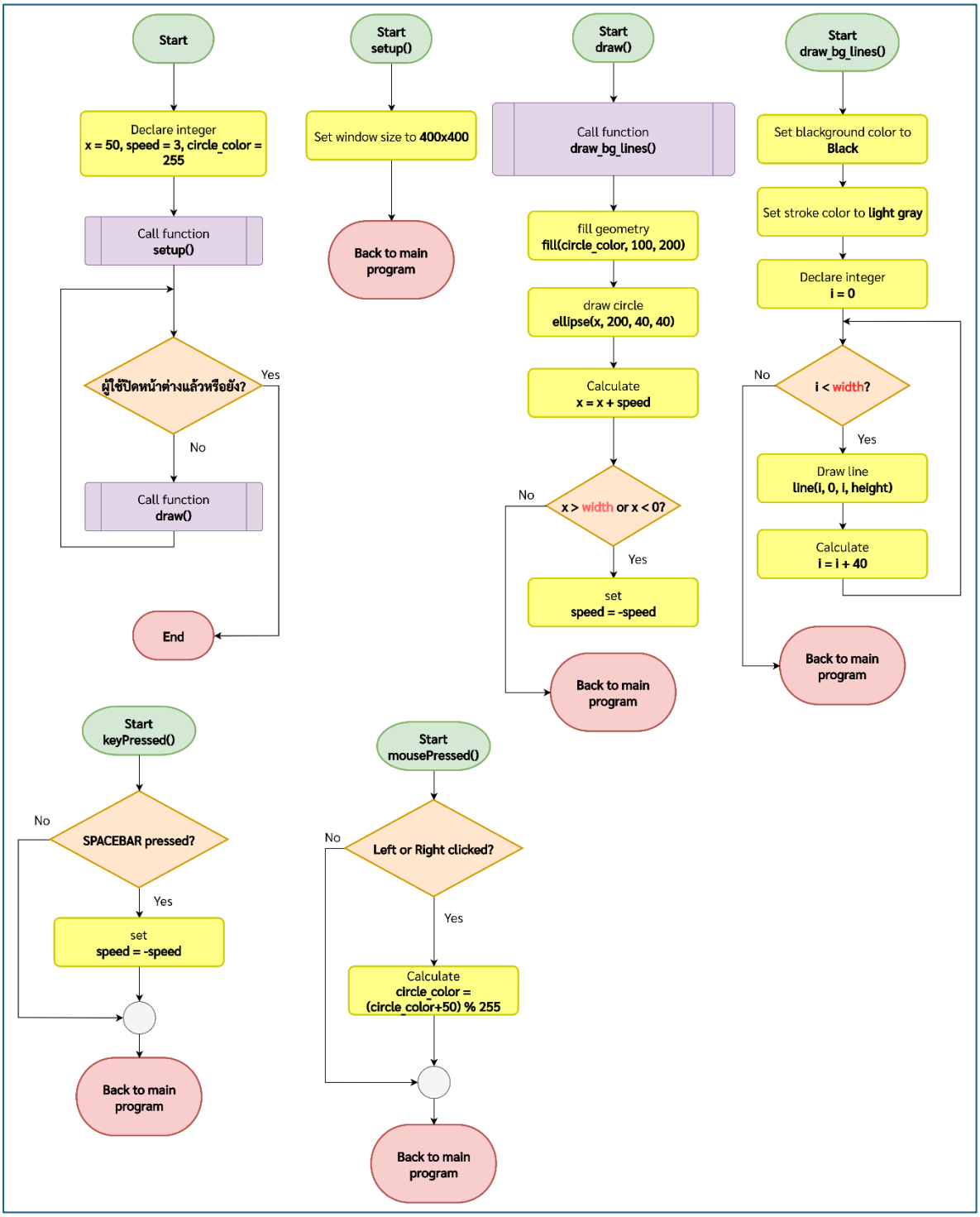
```
#Function that draws a single square
def draw_square(x, y, size_val):
    rect(x, y, size_val, size_val)

#Function that draws a plus sign using draw_square()
def draw_positive(x, y, size_val):
    draw_square(x, y, size_val)
    draw_square(x, y - size_val, size_val)
    draw_square(x, y + size_val, size_val)
    draw_square(x - size_val, y, size_val)
    draw_square(x + size_val, y, size_val)
```


Lab4_02

ข้อที่ 3 สร้าง Animation และ Interaction จากสิ่งที่นักศึกษาสนใจ

โปรแกรมลูกบอลเด้งที่ขอบ และสามารถเปลี่ยนสีได้เมื่อคลิกซ้ายหรือขวา เปลี่ยนทิศการเคลื่อนที่ของบอลด้วยปุ่ม Spacebar



```
x = 50
speed = 3
circle_color = 255

def setup():
    size(400, 400)

def draw_bg_lines():
    background(30)
    stroke(80)
    i = 0
    while i < width:
        line(i, 0, i, height)
        i += 40

def draw():
    global x, speed

    draw_bg_lines()

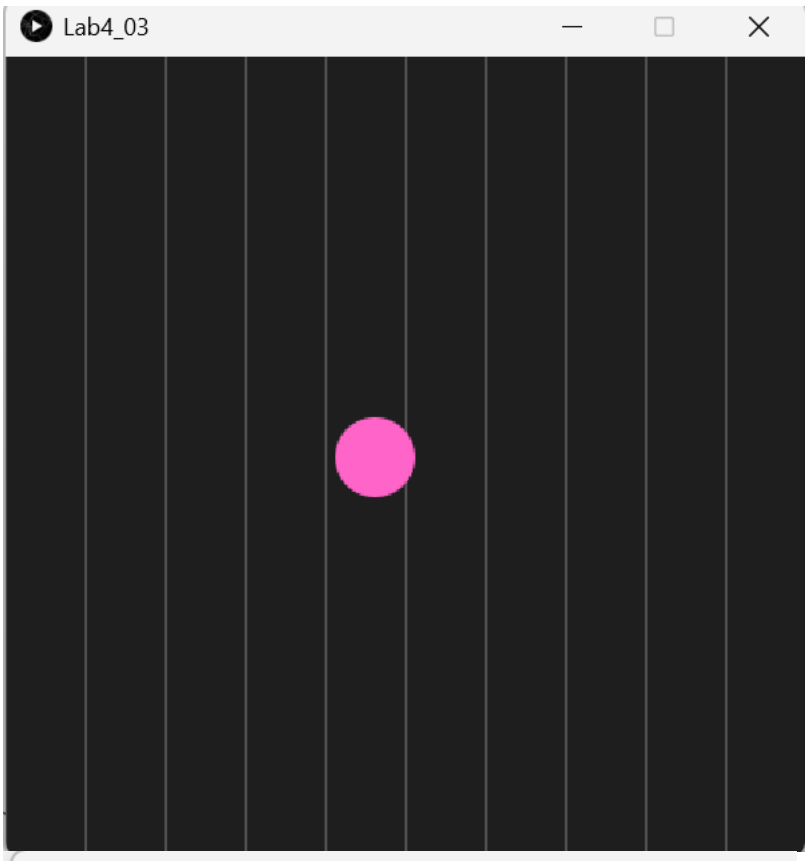
    fill(circle_color, 100, 200)
    noStroke()
    ellipse(x, 200, 40, 40)

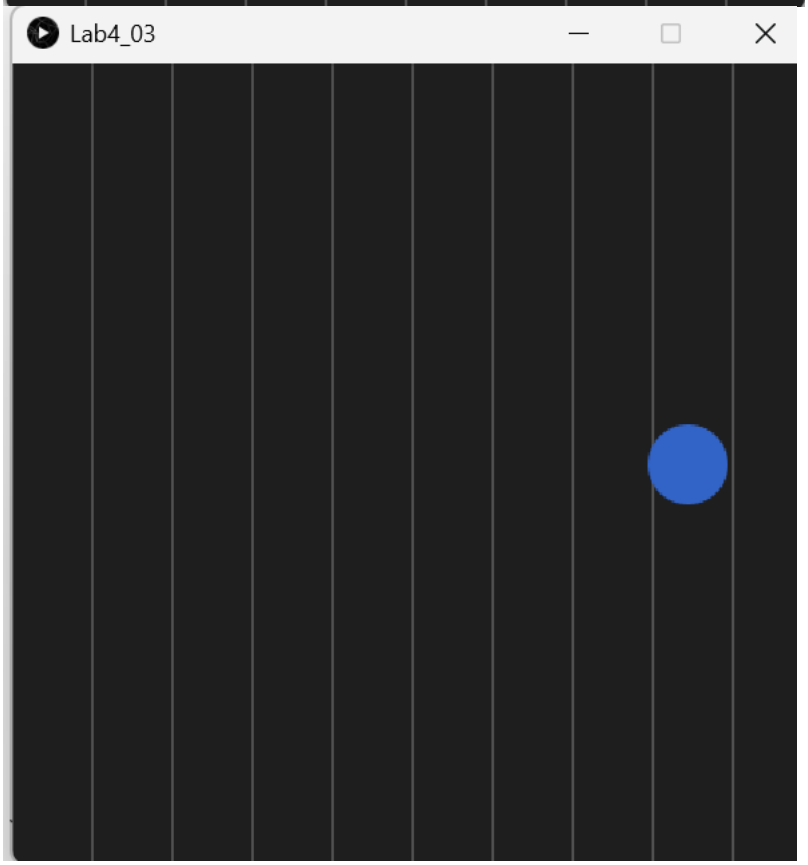
    x = x + speed

    if x > width or x < 0:
        speed = -speed

def mousePressed():
    global circle_color
    circle_color = (circle_color + 50) % 255

def keyPressed():
    global speed
    if key == ' ':
        speed = -speed
```

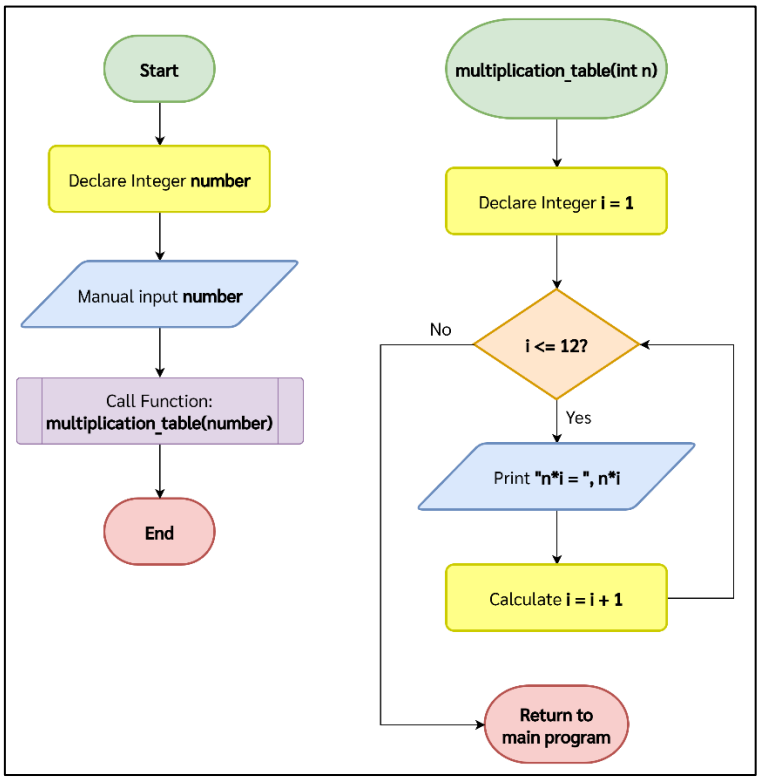

Lab4_03


Lab4_03

Text Mode

ข้อที่ 4 ตารางสูตรคูณ

โปรแกรมในข้อนี้จะเริ่มต้นโดยการรับค่าจำนวนเต็มเก็บในตัวแปร number แล้วจึงนำไปคำนวณใน user-defined function ที่ชื่อ multiplication_table() โดยมี parameter n ทำหน้าที่รับ argument ที่เป็นจำนวนเต็มเข้าไปใช้ในการทำตารางสูตรคูณ โดยในฟังก์ชันจะมีการกำหนดตัวนับที่เป็นจำนวนเต็ม i โดยเริ่มที่ 1 แล้วแสดงผลลัพธ์ของ n คูณกับ i และวนซ้ำจนมีค่าเท่ากับ 12 ดังที่จะได้แสดงให้เห็นจากผังงานด้านล่าง




```python
'''
    ข้อที่ 4 ตารางแม่สูตรคูณ
    6901012610544 นายสุทธิภัทร เอื้อพัฒนากิจ
'''

def multiplication_table(n):
    i = 1
    while i <= 12:
            print(f"{number} x {i} = {number*i}")
            i = i + 1

if __name__ == '__main__':
    number = int(input("Enther number you want to be shown in Multiplication Table >>"))
    multiplication_table(number)
```

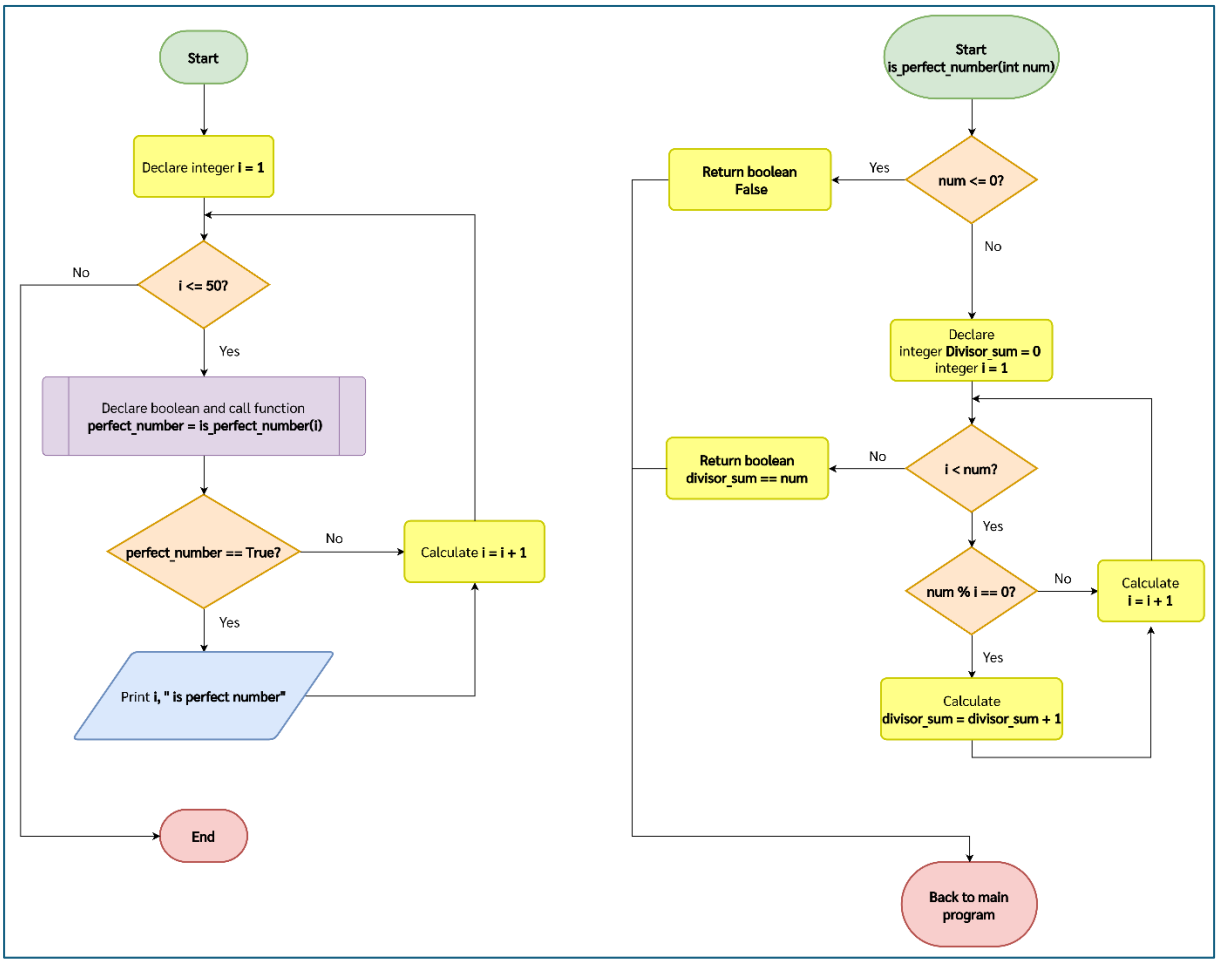
```
Enther number you want to be shown in Multiplication Table >>3
3 x 1 = 3
3 x 2 = 6
3 x 3 = 9
3 x 4 = 12
3 x 5 = 15
3 x 6 = 18
3 x 7 = 21
3 x 8 = 24
3 x 9 = 27
3 x 10 = 30
3 x 11 = 33
3 x 12 = 36
```

```
Enther number you want to be shown in Multiplication Table >>25
25 x 1 = 25
25 x 2 = 50
25 x 3 = 75
25 x 4 = 100
25 x 5 = 125
25 x 6 = 150
25 x 7 = 175
25 x 8 = 200
25 x 9 = 225
25 x 10 = 250
25 x 11 = 275
25 x 12 = 300
```

```
Enther number you want to be shown in Multiplication Table >>-4
-4 x 1 = -4
-4 x 2 = -8
-4 x 3 = -12
-4 x 4 = -16
-4 x 5 = -20
-4 x 6 = -24
-4 x 7 = -28
-4 x 8 = -32
-4 x 9 = -36
-4 x 10 = -40
-4 x 11 = -44
-4 x 12 = -48
```

ข้อที่ 5 Perfect Number

วิธีการหา Perfect Number ทั้งหมดตั้งแต่ 1 ถึง 50 โดยอาศัยการวนซ้ำนั้นมีวิธีการคือ เบื้องต้น ถ้าตัวเลข (num) น้อยกว่าหรือเท่ากับ 0 จะส่งกลับ False ทันที เพราะ Perfect Number ต้องเป็นจำนวนเต็มบวกเท่านั้น ต่อมาจะหาตัวหารและผลบวก ใช้ลูป while วนหาเลขตั้งแต่ 1 ถึง num - 1 โดยใช้ตัวดำเนินงาน % (Modulo) เพื่อเช็กว่า num % i == 0 หรือไม่ ถ้าหารลงตัว แสดงว่า i เป็นตัวหารแท้ จะนำค่า i นั้นไปบวกสะสมไว้ใน divisor_sum แล้วจึงเปรียบเทียบผลลัพธ์ เมื่อวนลูปครบแล้ว จะเช็กว่า divisor_sum == num หรือไม่ ถ้าผลบวกตัวหารเท่ากับเลขเดิม จะคืนค่า True ถ้าไม่เท่าจะคืนค่า False สุดท้ายวนลูปทดสอบช่วงตัวเลข ในส่วน Main program จะใช้ลูป while ส่งเลข 1 ถึง 50 เข้าไปตรวจในฟังก์ชันทีละตัว และพิมพ์เฉพาะตัวเลขที่ได้ผลลัพธ์เป็น True ออกมา



```python
def is_perfect_number(num):
    if num <= 0:
        return False

    divisor_sum = 0
    i = 1

    while i < num:
        if num % i == 0:
            divisor_sum += i
        i += 1
    return divisor_sum == num


if __name__ == '__main__':
    #number = int(input())
    i = 1
    while i <= 50:
        perfect_num = is_perfect_number(i)
        if perfect_num:
                print(f"{i} is perfect number.")
        i = i + 1
```

```
python.exe d:/KMUTNB/1-69/Computer-Programming/week4/Lab/Lab4-05.py
6 is perfect number.
28 is perfect number.
```

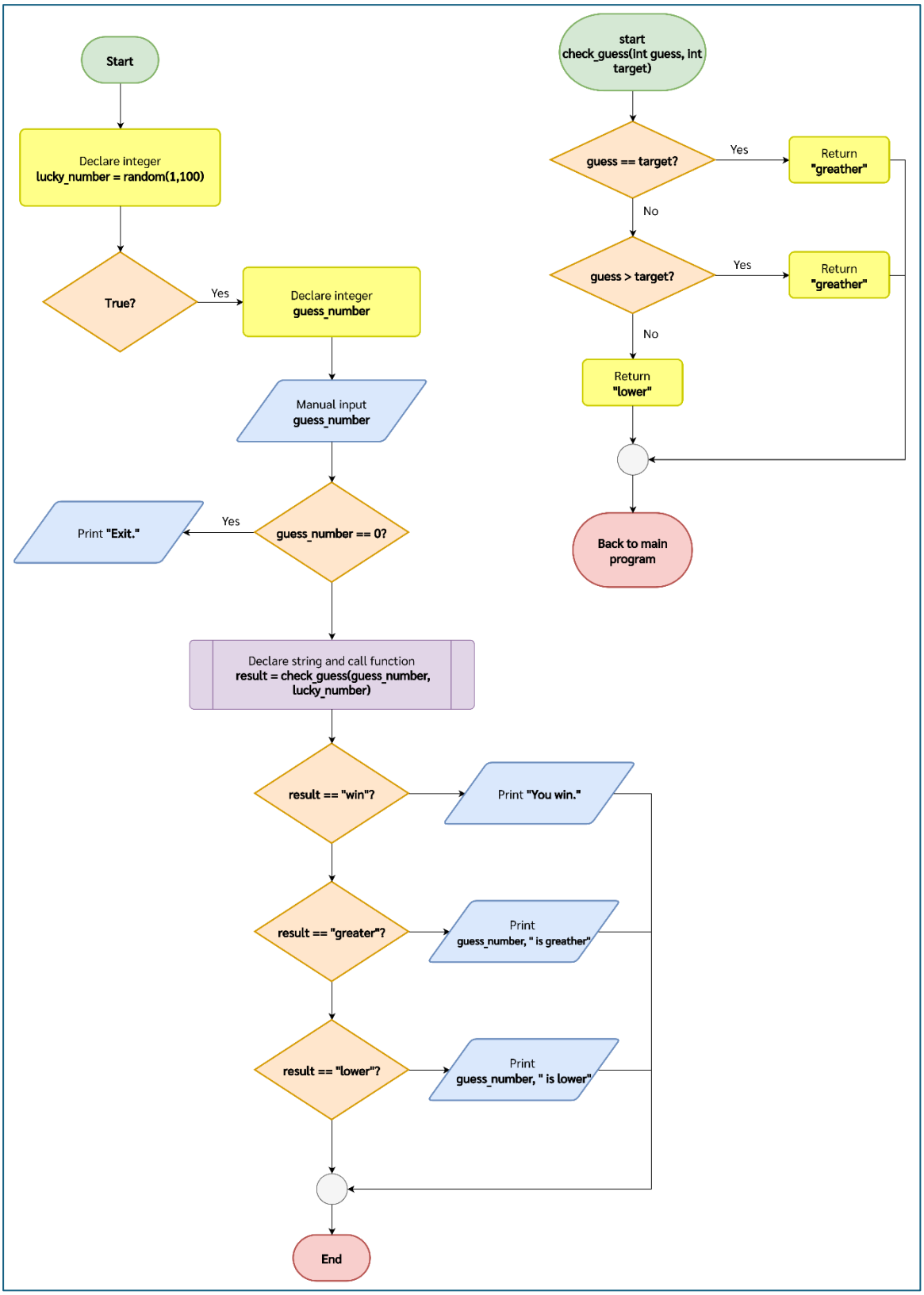
ข้อที่ 6 เกมทายตัวเลข

เกมทายตัวเลขสุ่ม ซึ่งเริ่มต้นด้วยการสร้างตัวเลขเป้าหมายแบบจำนวนเต็มในช่วง 1 ถึง 100 ขึ้นมาหนึ่งตัวเก็บไว้ในตัวแปร lucky_number โดยอาศัยฟังก์ชันจากไลบรารี random จากนั้น โปรแกรมจะเข้าสู่ลูปวนซ้ำแบบไม่สิ้นสุด เพื่อให้ผู้ใช้ได้ป้อนตัวเลขทายผลเข้ามาเรื่อยๆ

ในการทำงานแต่ละรอบ โปรแกรมจะรับค่าจากผู้ใช้เป็นจำนวนเต็มเก็บไว้ในตัวแปร guess_number แล้วตรวจสอบเงื่อนไขแรกทันทีว่าผู้ใช้ป้อนเลข 0 เข้ามาหรือไม่ หากป้อนเลข 0 โปรแกรมจะถือว่าผู้ใช้ต้องการออกจากเกม จึงพิมพ์ข้อความแจ้งเตือน "Exit." แล้วใช้คำสั่ง break เพื่อหยุดลูปและจบการทำงานทันที

หากตัวเลขที่ป้อนเข้ามาไม่ใช่ 0 โปรแกรมจะนำค่านั้นไปส่งต่อให้ฟังก์ชัน check_guess() เพื่อเปรียบเทียบกับเลขเป้าหมาย โดยฟังก์ชันจะคืนค่าคำตอบเป็นข้อความเพื่อบอกสถานะ ได้แก่ "win" เมื่อทายถูกต้อง "greater" เมื่อทายสูงเกินไป และ "lower" เมื่อทายน้อยเกินไป

สุดท้าย โปรแกรมจะนำผลลัพธ์ที่ได้จากฟังก์ชันมาแสดงคำแนะนำให้ผู้ใช้ทราบ หากผลออกมาเป็น "win" โปรแกรมจะพิมพ์ "You win." แล้วจบลูปการทำงานทันที แต่หากทายผิด โปรแกรมจะพิมพ์บอกว่าตัวเลขที่ป้อนเข้ามานั้นมากกว่าหรือน้อยกว่าตัวเลขเป้าหมาย เพื่อให้ผู้ใช้เก็บเป็นข้อมูลในการวนลูปทายตัวเลขครั้งถัดไปจนกว่าจะถูกต้อง


Start
Declare integer
lucky_number = random(1,100)
True?
Yes
Declare integer
guess_number
Manual input
guess_number
guess_number == 0?
Yes
Print "Exit."
Declare string and call function
result = check_guess(guess_number,
lucky_number)
result == "win"?
Print "You win."
result == "greater"?
Print
guess_number, " is greather"
result == "lower"?
Print
guess_number, " is lower"
End
start
check_guess(int guess, int
target)
guess == target?
Yes
Return
"greather"
No
guess > target?
Yes
Return
"greather"
No
Return
"lower"
Back to main
program

```
Guess the number >> 25
25 is lower than the lucky number.
Guess the number >> 30
30 is lower than the lucky number.
Guess the number >> 33
33 is lower than the lucky number.
Guess the number >> 31
31 is lower than the lucky number.
Guess the number >> 32
32 is lower than the lucky number.
Guess the number >> 33
33 is lower than the lucky number.
Guess the number >> 34
You win.
```

```
Guess the number >> 33
33 is lower than the lucky number.
Guess the number >> 55
55 is greater than the lucky number.
Guess the number >> 66
66 is greater than the lucky number.
Guess the number >> 0
Exit.
```

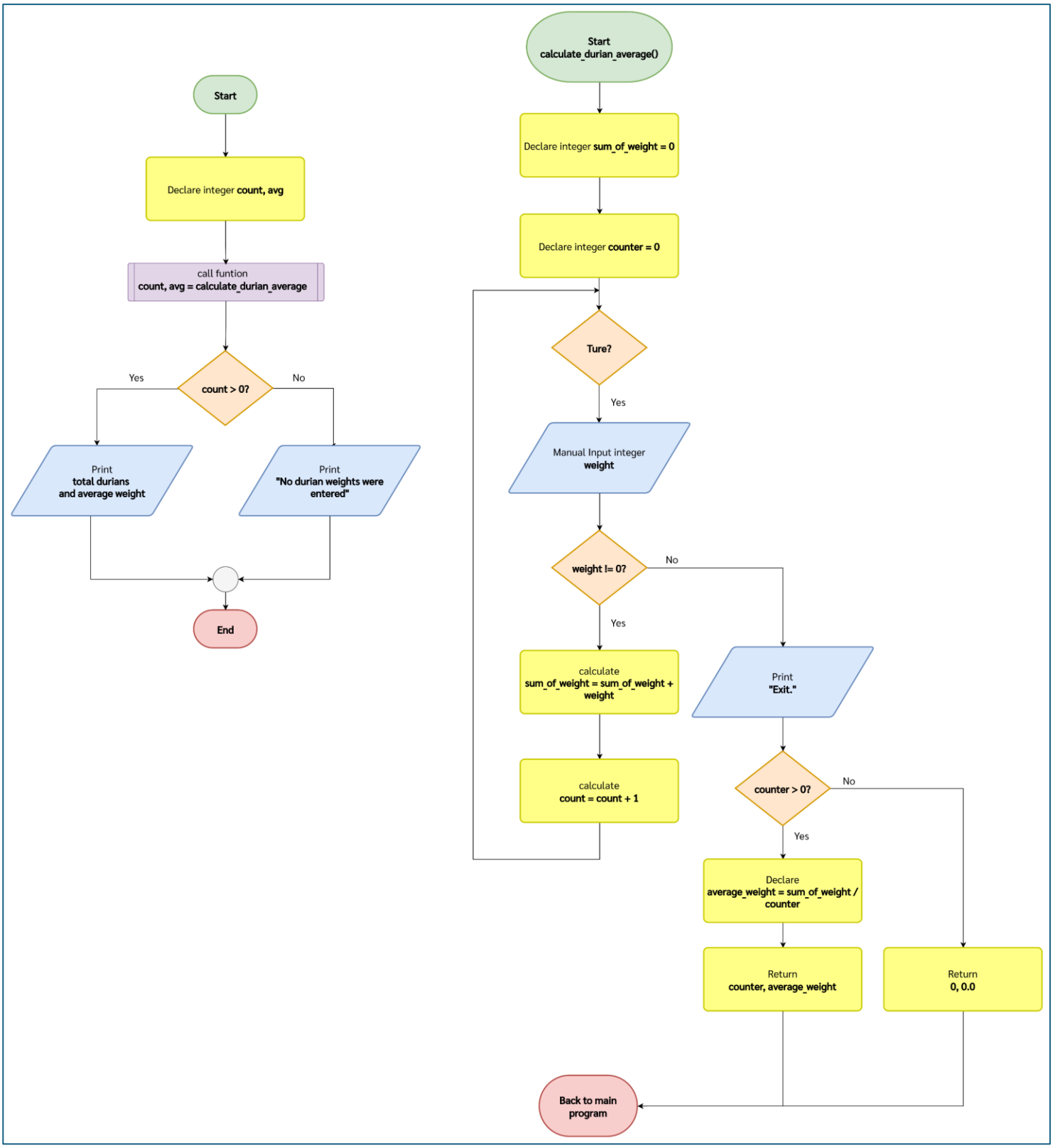
ข้อที่ 7 นำหนักเฉลี่ยของทุเรียน

สำหรับโปรแกรมคำนวณน้ำหนักเฉลี่ยของทุเรียน โดยเริ่มทำงานในฟังก์ชัน calculate_durian_average() ด้วยการตั้งตัวแปรเก็บผลรวมน้ำหนัก sum_of_weight และตัวนับจำนวนทุเรียน counter ให้มีค่าเป็น 0 จากนั้นจะเข้าสู่ลูปวนรับค่าน้ำหนักทุเรียนจากผู้ใช้ทางคีย์บอร์ดในรูปแบบจำนวนเต็ม

สำหรับการทำงานภายในลูป หากผู้ใช้ป้อนค่าน้ำหนักทุเรียนที่ไม่ใช่ 0 โปรแกรมจะนำค่าน้ำหนักนั้นไปบวกสะสมไว้ใน sum_of_weight และเพิ่มตัวนับ counter ขึ้นทีละ 1 แต่ถ้าผู้ใช้ป้อนเลข 0 เข้ามา โปรแกรมจะถือว่าเป็นคำสั่งจบการทำงาน จึงพิมพ์ข้อความ "Exit." และใช้คำสั่ง break เพื่อออกจากลูปวนรับค่าทันที

เมื่อออกจากลูปแล้ว ฟังก์ชันจะเช็กเงื่อนไขว่ามีการป้อนน้ำหนักเข้ามาบ้างหรือไม่ หากมีทุเรียนอย่างน้อย 1 ลูก (counter > 0) จะทำการคำนวณหาน้ำหนักเฉลี่ยด้วยการนำผลรวมน้ำหนักหารด้วยจำนวนทุเรียน แล้วส่งคืนค่าจำนวนทุเรียนและค่าน้ำหนักเฉลี่ยกลับออกไป แต่ถ้าไม่มีการป้อนน้ำหนักเลย ฟังก์ชันจะส่งคืนค่า 0 และ 0.0 กลับไปแทน

สุดท้าย ในส่วน Main program จะรับค่าจากฟังก์ชันมาเก็บไว้ในตัวแปร count และ avg หากรับค่ามาแล้วพบว่า count มากกว่า 0 โปรแกรมจะพิมพ์สรุปจำนวนทุเรียนทั้งหมดพร้อมค่าน้ำหนักเฉลี่ยที่เป็นทศนิยม 2 ตำแหน่งออกมา แต่ถ้าไม่มีการระบุน้ำหนักใดๆ ตั้งแต่แรก ก็จะแสดงข้อความแจ้งว่าไม่มีข้อมูลน้ำหนักทุเรียนถูกป้อนเข้ามา


Start
Declare integer count, avg
call funtion
count, avg = calculate_durian_average
count > 0?
Yes
No
Print
total durians
and average weight
Print
"No durian weights were
entered"
End
Start
calculate_durian_average()
Declare integer sum_of_weight = 0
Declare integer counter = 0
Ture?
Yes
Manual Input integer
weight
weight != 0?
No
Yes
calculate
sum_of_weight = sum_of_weight +
weight
calculate
count = count + 1
Print
"Exit."
counter > 0?
No
Yes
Declare
average_weight = sum_of_weight /
counter
Return
counter, average_weight
Return
0, 0.0
Back to main
program

```python
def calculate_durian_average():
    sum_of_weight = 0
    counter = 0

    while True:
        weight = int(input("Please input Durian weight: "))
        if weight != 0:
            sum_of_weight += weight
            counter += 1
        else:
            print("Exit.")
            break

    if counter > 0:
        average_weight = sum_of_weight / counter
        return counter, average_weight
    else:
        return 0, 0.0


if __name__ == '__main__':
    count, avg = calculate_durian_average()

    if count > 0:
        print(f"Average weight of {count} durians is {avg:.2f} kg.")
    else:
        print("No durian weights were entered.")
```

```
Please input Durian weight: 1
Please input Durian weight: 2
Please input Durian weight: 3
Please input Durian weight: 0
Exit.
Average weight of 3 durians is 2.00 kg.
```

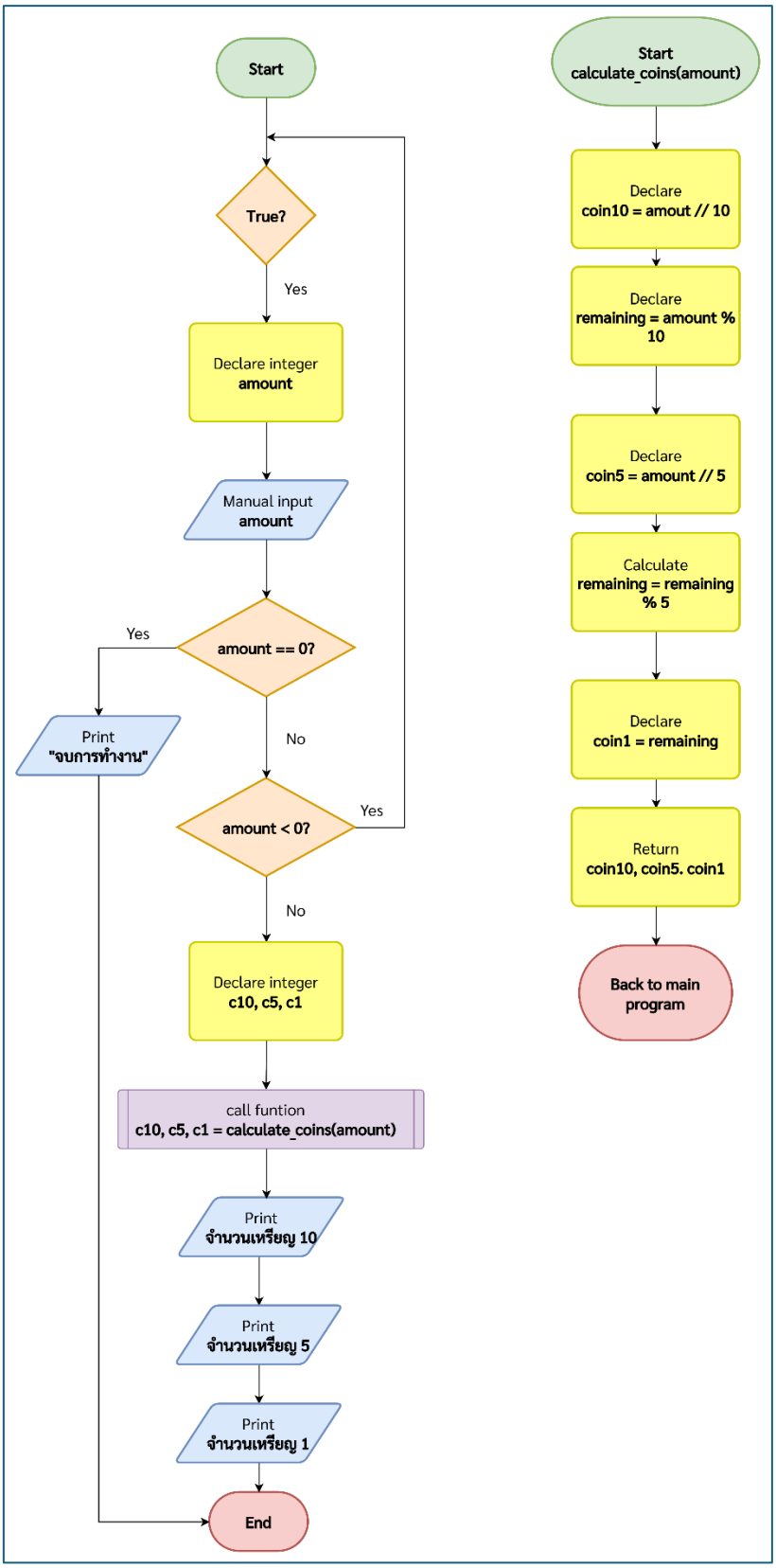
```
Please input Durian weight: 0
Exit.
No durian weights were entered.
```

ข้อที่ 8 แลกเหรียญ

การแยกจำนวนเหรียญทอน (เหรียญ 10, 5 และ 1 บาท) ตามยอดเงินที่ป้อนเข้ามา เริ่มโดยมีฟังก์ชัน calculate_coins() ทำหน้าที่คำนวณหาจำนวนเหรียญโดยเริ่มจากการใช้ตัวหารเอาส่วน (//) ด้วย 10 เพื่อหาจำนวนเหรียญ 10 บาทที่ต้องใช้ แล้วใช้ตัวหารเอาเศษ (%) หาเงินที่เหลือ นำเศษนั้นมาหาจำนวนเหรียญ 5 บาทด้วยวิธีเดียวกัน และเศษสุดท้ายที่เหลือทั้งหมดจะกลายเป็นจำนวนเหรียญ 1 บาท ก่อนจะส่งคืนค่าจำนวนเหรียญทั้งสามชนิดกลับไป

ในส่วน Main program จะใช้ลูป while True เพื่อรับการป้อนจำนวนเงินจากผู้ใช้เรื่อยๆ พร้อมระบบตรวจสอบข้อมูล โดยหากป้อนเลข 0 โปรแกรมจะพิมพ์ข้อความแจ้งเตือนจบการทำงานและออกจากลูปทันที แต่หากป้อนค่าติดลบ โปรแกรมจะแจ้งเตือนให้ระบุจำนวนเงินที่มากกว่าหรือเท่ากับ 0 แล้วใช้คำสั่ง continue เพื่อข้ามไปเริ่มรับค่ารอบถัดไปทันที

หากป้อนจำนวนเงินเป็นจำนวนเต็มบวก โปรแกรมจะส่งค่าไปคำนวณในฟังก์ชันแล้วรับผลลัพธ์กลับมาพิมพ์แสดงจำนวนเหรียญ 10 บาท 5 บาท และ 1 บาทตามลำดับ ก่อนจะวนกลับไปรอรับค่าในรอบถัดไปอย่างต่อเนื่อง


Start
True?
Yes
Declare integer
amount
Manual input
amount
amount == 0?
Yes
Print
"จบการทำงาน"
No
amount < 0?
Yes
No
Declare integer
c10, c5, c1
call funtion
c10, c5, c1 = calculate_coins(amount)
Print
จำนวนเหรียญ 10
Print
จำนวนเหรียญ 5
Print
จำนวนเหรียญ 1
End
Start
calculate_coins(amount)
Declare
coin10 = amout // 10
Declare
remaining = amount %
10
Declare
coin5 = amount // 5
Calculate
remaining = remaining
% 5
Declare
coin1 = remaining
Return
coin10, coin5. coin1
Back to main
program

```python
def calculate_coins(amount):
    coin10 = amount // 10
    remaining = amount % 10

    coin5 = remaining // 5
    remaining = remaining % 5

    coin1 = remaining

    return coin10, coin5, coin1


if __name__ == '__main__':
    while True:
        amount = int(input("ใส่จำนวนเงินที่ต้องการแลก (ใส่ 0 เพื่อหยุดทำงาน): "))

        if amount == 0:
            print("จบการทำงานของโปรแกรม")
            break

        if amount < 0:
            print("กรุณาใส่จำนวนเงินที่มากกว่าหรือเท่ากับ 0")
            continue

        c10, c5, c1 = calculate_coins(amount)

        #Shows result
        print(f"เหรียญ 10 จำนวน {c10} เหรียญ")
        print(f"เหรียญ 5 จำนวน {c5} เหรียญ")
        print(f"เหรียญ 1 จำนวน {c1} เหรียญ")
        print("-" * 30)
```

```
ใส่จำนวนเงินที่ต้องการแลก (ใส่ 0 เพื่อหยุดทำงาน): 28
เหรียญ 10 จำนวน 2 เหรียญ
เหรียญ 5 จำนวน 1 เหรียญ
เหรียญ 1 จำนวน 3 เหรียญ
------------------------------
ใส่จำนวนเงินที่ต้องการแลก (ใส่ 0 เพื่อหยุดทำงาน): 50
เหรียญ 10 จำนวน 5 เหรียญ
เหรียญ 5 จำนวน 0 เหรียญ
เหรียญ 1 จำนวน 0 เหรียญ
------------------------------
ใส่จำนวนเงินที่ต้องการแลก (ใส่ 0 เพื่อหยุดทำงาน): 0
จบการทำงานของโปรแกรม
```